DHA SUFFA UNIVERSITY

Department of Computer Science

Artificial Intelligence Program

CS-2001L

Fall 2026

LAB 01

Introduction to Python

Objectives:

Introduction to Artificial Intelligence paradigm
Using python programming language
Setting up the lab environment (Google Colab / Python Installation)
Basics of python (if-else, loops, functions)

Artificial Intelligence:

Artificial Intelligence (AI) is a branch of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence. These tasks include learning, reasoning, problem solving, perception, and decision making. AI systems analyze data, identify patterns, and make predictions or decisions based on algorithms and models. AI is widely used in modern applications such as virtual assistants, recommendation systems, autonomous vehicles, healthcare diagnostics, and robotics. The goal of AI is to develop machines that can simulate intelligent behavior and improve performance over time through experience.

Python:

Python is a high-level, interpreted programming language known for its simplicity and readability. It is widely used in artificial intelligence, machine learning, data science, web development, and automation.
Python is popular in AI development because it provides powerful libraries such as NumPy, Pandas, TensorFlow, and PyTorch that simplify complex computations and model building. Python's syntax is easy to learn, making it an ideal programming language for beginners and researchers.

Key features of Python include:

- Simple and readable syntax
- Large standard library and community support
- Platform independence
- Support for multiple programming paradigms (procedural, object-oriented, and
- functional)

Installation:

Go through this YouTube video to locally install python in your PC
https://youtu.be/3E3NZRpXZyI

After successful installation, open CMD and check version of python using command
python –version or py --version

Setting up python environment:

Create python's virtual environment using command
python –m venv first-venv or py –m venv first-venv

To create with a specific version, use this command: py -3.xx –m venv my-venv

Activate the virtual environment
first-venv\Scripts\activate

To deactivate
deactivate

Google Colab

Google Colaboratory (Colab) is a cloud-based platform provided by Google that allows users to write and execute Python code in a browser. It is particularly useful for AI and machine learning experiments because it provides free access to GPUs and requires no local installation.
Benefits of Google Colab include:

- No software installation required
- Cloud storage integration with Google Drive
- Free GPU and TPU support
- Easy sharing and collaboration

Stage to run the code in Python

Simple Program in Python

In this page, we will learn how to write the simple program of Python. We can write a simple hello Python program easily after installing

Basics of python

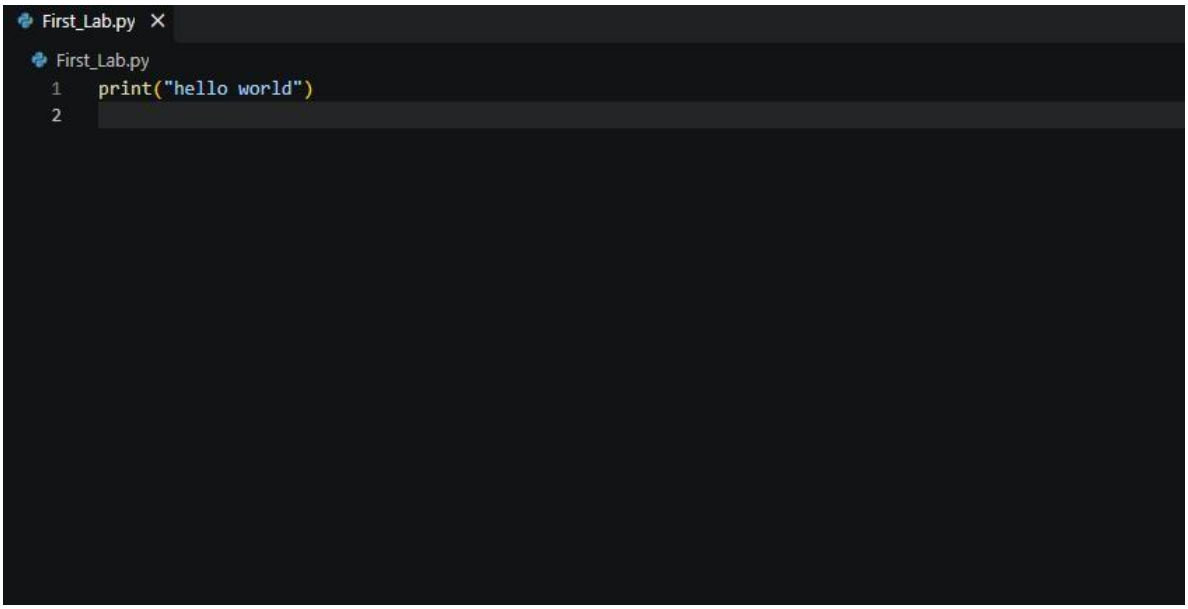
1. Print Statement

The print() function is used to display output in Python.
Multiple values can also be printed using commas or formatted strings.
Example: print("Hello, world!")

Creating hello Python example

Let's create the hello Python program:

To write the simple program, open VS-> write simple program as displayed below:



Output:



Variables

Variables are used to store data values in a program. In Python, variables are created when a value is assigned to them. Python does not require explicit declaration of variable types because it is dynamically typed.

Example: num = 4
name = “Mike”

Data types:

A variable is a named box that stores a value. You create one by writing name = value

Type	Meaning	Business example
`int`	whole number	quantity = 4
`float`	decimal number	price = 150.5
`str`	text	product = "Notebook"
`bool`	True or False	in_stock = True

Example code of variable and data types

First_Lab.py

```
1   product = "Notebook"
2   price = 150.5
3   quantity = 4
4   in_stock = True
5
6   print(product, price, quantity, in_stock)
7   print(type(product), type(price), type(quantity), type(in_stock))
8
```

Output

```
PS C:\Users\sahil\OneDrive\Desktop\for_python> python First_Lab.py
Notebook 150.5 4 True
<class 'str'> <class 'float'> <class 'int'> <class 'bool'>
PS C:\Users\sahil\OneDrive\Desktop\for_python>
```

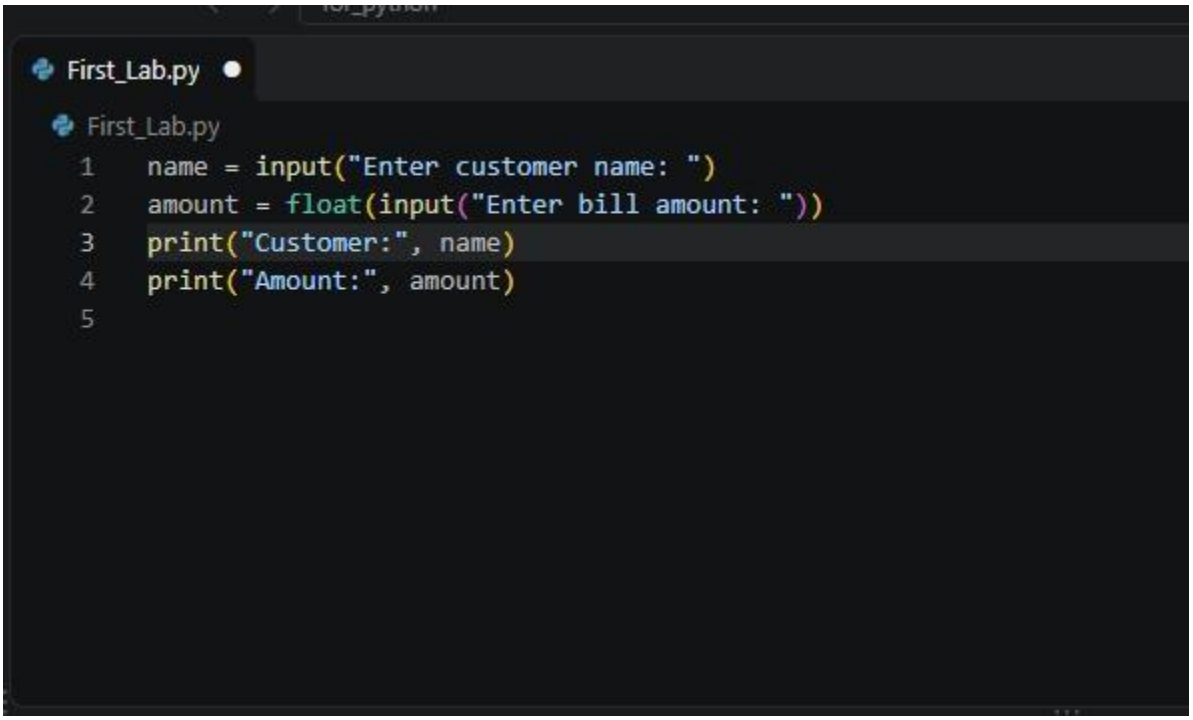
Line-by-line:

- Lines 1 to 4 create four variables. Python decides the type from the value, so you do not declare it.
- `print(product, price, quantity, in_stock)` shows the four values.
- `type(x)` tells you the type of `x`.

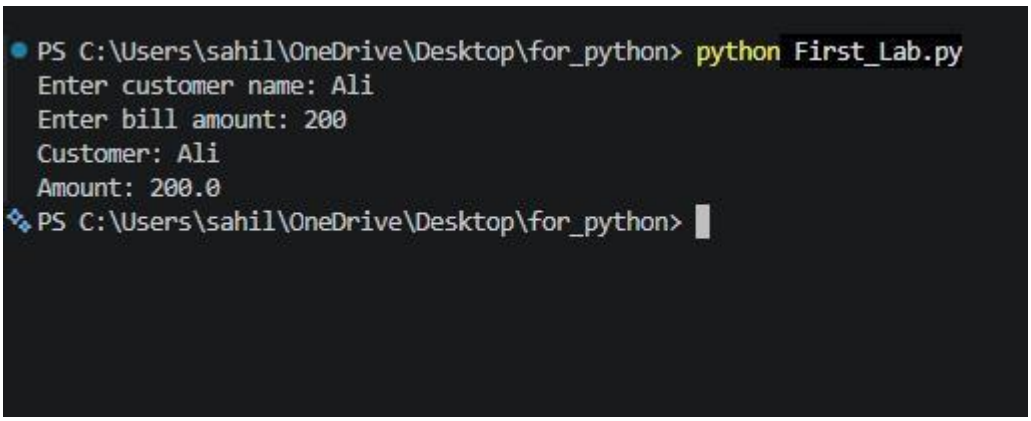
Naming rules: use letters, digits and underscores; do not start with a digit; no spaces; names are case-sensitive (`Price` and `price` are different). Use meaningful names.

Input and type conversion :

input() reads what the user types. It always returns a string, even if the user types a number. To do arithmetic you must convert it with int() or float().



Output:



Operators:

1. Arithmetic Operators

These operators perform basic mathematical operations:

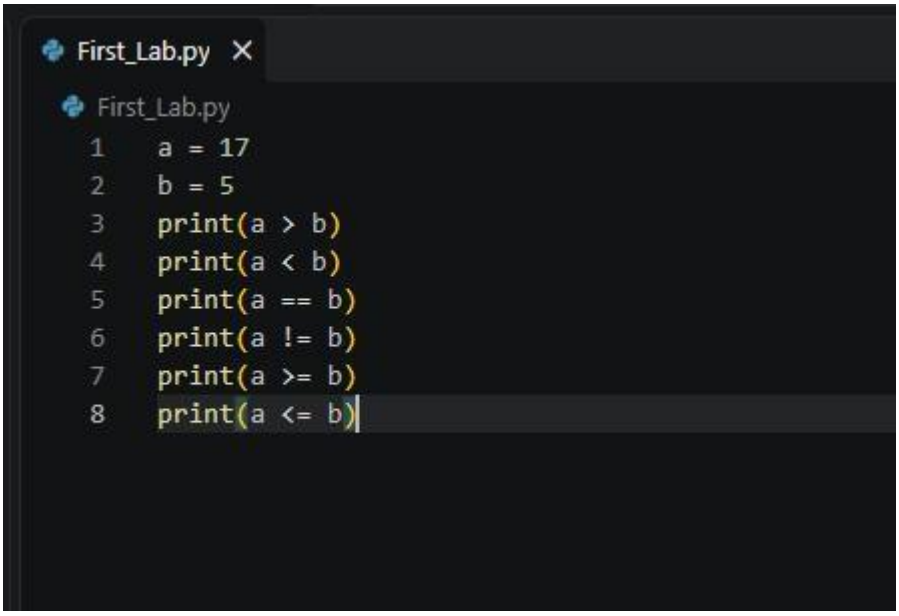
- **Addition (+)**: Adds two operands.
  Example: a + b
- **Subtraction (-)**: Subtracts the second operand from the first.
  Example: a - b
- **Multiplication (*)**: Multiplies two operands.
  Example: a * b
- **Division (/)**: Divides the first operand by the second.
  Example: a / b
- **Modulus (%)**: Returns the remainder of a division operation.
  Example: a % b

2. Relational Operators

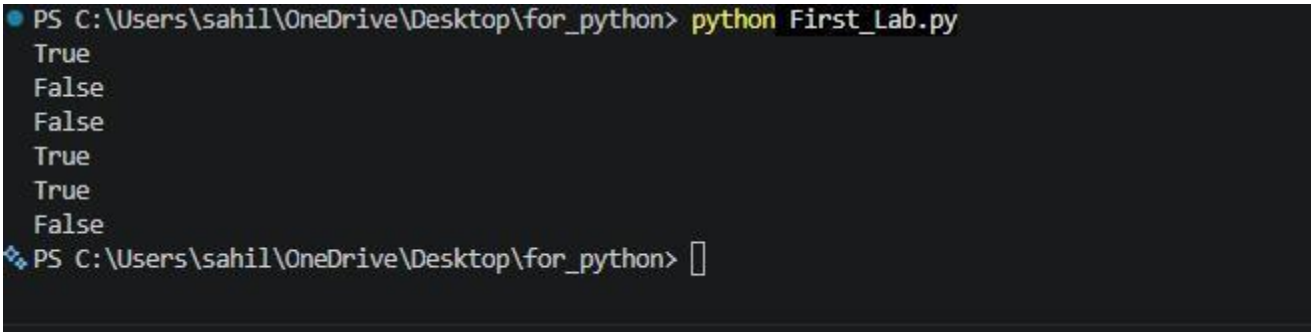
These operators compare two values:

- **Equal to (==)**: Checks if two operands are equal.
  Example: a == b
- **Not equal to (!=)**: Checks if two operands are not equal.
  Example: a != b
- **Greater than (>)**: Checks if the left operand is greater than the right.
  Example: a > b
- **Less than (<)**: Checks if the left operand is less than the right.
  Example: a < b

- **Greater than or equal to (>=)**: Checks if the left operand is greater than or equal to the right.
  Example: a >= b
- **Less than or equal to (<=)**: Checks if the left operand is less than or equal to the right.
  Example: a <= b



Output



3. Logical Operators

These operators are used to perform logical operations:

- **Logical AND (and)**: Returns true if both operands are true.
  Example: a and b
- **Logical OR (or)**: Returns true if at least one of the operands is true.
  Example: a or b
- **Logical NOT (not)**: Reverses the logical state of its operand.
  Example: not a

```python
age = 25
has_id = True
print(age >= 18 and has_id)
print(age >= 18 or has_id)
print(not has_id)
```

Output

```
False
PS C:\Users\sahil\OneDrive\Desktop\for_python> python First_Lab.py
True
True
False
PS C:\Users\sahil\OneDrive\Desktop\for_python>
```

4. Assignment Operators

These operators assign values to variables:

- **Simple assignment (=)**: Assigns the right operand to the left.
  Example: a = b
- **Add and assign (+=)**: Adds the right operand to the left operand and assigns the result to the left.
  Example: a += b (equivalent to a = a + b)
- **Subtract and assign (-=)**: Subtracts the right operand from the left and assigns the result.
  Example: a -= b
- **Multiply and assign (*=)**: Multiplies the left operand by the right and assigns the result.
  Example: a *= b
- **Divide and assign (/=)**: Divides the left operand by the right and assigns the result.
  Example: a /= b

First_Lab.py

```python
total = 100
total += 50
total -= 20
total *= 2
total /= 4
print(total)
```

Output

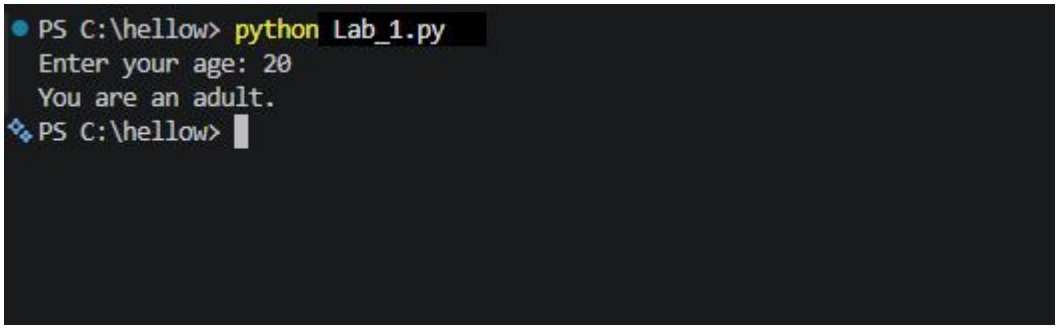
```
PS C:\Users\sahil\OneDrive\Desktop\for_python> python First_Lab.py
65.0
PS C:\Users\sahil\OneDrive\Desktop\for_python>
```

If - else Statement

Lab_1.py

```python
age = int(input("Enter your age: "))
if age < 18:
    print("You are a minor.")
else:
    print("You are an adult.")
```

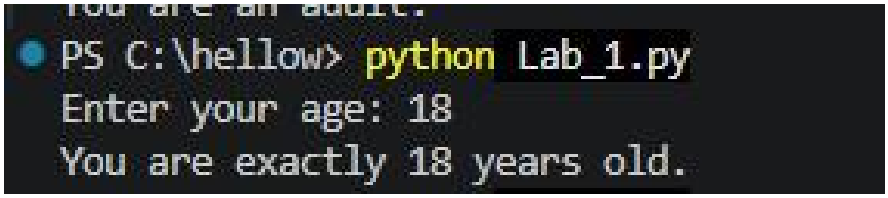
Output



If - else -elif statement



Output



Lab Tasks

1. declare one variable each of type int, float, str and bool to describe a product (name, price, quantity, availability). Print the values and their types

2. Write a python program that takes two numbers as input and performs addition, subtraction, multiplication, division, and modulus operations.

3. read the prices of two products and print the result of all six relational operators.

4. Implement a program that takes two boolean values as input and uses logical operators to display the results.

5. Write a python program to create a simple calculator
6. Write a program that takes a single integer from the user and checks two things:

   I) Whether the number is Positive, Negative, or Zero.
   II) If the number is Positive, check whether it is Even or Odd.